NOTE As I discussed in chapter 9, the dependencies of a service must always have a lifetime that's the same as or longer than that of the service itself, to avoid captive dependencies.

Imagine a slight variation on the caching example from section 34.1.1. Instead of storing the exchange rates in a singleton cache object, you want to save the exchange rates to a database so you can look up the historic rates.

Most database providers, including EF Core's `DbContext`, register their services with scoped lifetimes. That means you need to access the scoped `DbContext` from inside the singleton `ExchangeRatesHostedService`, which precludes injecting the `DbContext` with constructor injection. The solution is to create a new container scope every time you update the exchange rates.

In typical ASP.NET Core applications, the framework creates a new container scope every time a new request is received, immediately before the middleware pipeline executes. All the services that are used in that request are fetched from the scoped container. When the request ends, the scoped container is disposed, along with any of the `IDisposable` scoped and transient services that were obtained from it. In a background service, however, there *are* no requests, so no container scopes are created. The solution is to create your own.

You can create a new container scope anywhere you have access to an `IServiceProvider` by calling `IServiceProvider.CreateScope()`. This creates a scoped container, which you can use to safely retrieve scoped and transient services.

WARNING Always make sure to dispose of the `IServiceScope` returned by `CreateScope()` when you're finished with it, typically